

Cesium Sandcastle Prompt Cheat Sheet Mini Lab 1

How to use this: **Always include prompt 0**; append the next prompt to the previous conversation instead of starting over.

Prompt 0 – Global Setup

Paste this prompt at the top of every prompt, every time.

You are helping me write Cesium Sandcastle code.

Technical requirements:

- Code must be completely in JavaScript (Cesium Sandcastle only accepts JS)
- Do NOT provide HTML files
- Any UI panels, checkboxes or buttons must be created dynamically using JavaScript
- Use robust error handling where appropriate
- Use console.log statements for important actions and state changes
- Code must be copy-paste safe for Cesium Sandcastle
- Organise the code into clearly numbered sections with comments
- Do not remove or rewrite existing sections unless explicitly asked
- Reuse existing variables wherever possible
- Keep the code modular and readable

Prompt 1 – Viewer + Site Boundary

Create a Cesium Viewer application.

Requirements:

- Set Cesium.Ion.defaultAccessToken at the top ([insert your token](#))
- Create a Cesium Viewer using World Terrain
- Disable:
 - timeline
 - animation
 - infoBox
 - selectionIndicator
- Enable globe lighting
- Disable depthTestAgainstTerrain

Then add a project site boundary.

Boundary requirements:

- I will provide a GeoJSON polygon (**insert geojson**)
- Display it as a thick red outline

- No fill colour
- Raise the polygon slightly above the terrain so it remains visible
- Zoom the camera to the boundary once loaded
- Log successful loading of the viewer and boundary

Create:

Section 1 - Access Token

Section 2 - Viewer Setup

Section 3 - Site Boundary

Do not add any other features.

This is my geojson boundary:

```
const geojson = {
  type: "FeatureCollection",
  features: [{
    type: "Feature",
    geometry: {
      type: "Polygon",
      coordinates: [[
        [11.18837119132374, 48.08428289693569],
        [11.18837119132374, 48.082448398234845],
        [11.198191497954355, 48.082448398234845],
        [11.198191497954355, 48.08428289693569],
        [11.18837119132374, 48.08428289693569]
      ]]
    }
  }]
};
```

Prompt 2 – Add OSM Buildings

Extend the existing Cesium Sandcastle application.

Add OpenStreetMap buildings.

Requirements:

- Use Cesium.createOsmBuildingsAsync()
- Add OSM Buildings as a scene primitive
- Store a reference to the buildings for future visibility control
- Set building visibility to OFF by default
- Log successful loading and any errors

Create this as:

Section 4 - OSM Buildings

Do not modify or rewrite existing sections.

Prompt 3 – Add Photogrammetry

Extend the existing Cesium Sandcastle application.

Add a photogrammetry tileset from Cesium ion.

Requirements:

- Load a Cesium ion 3D Tiles asset using an asset ID (**insert your asset ID**)
- Store a reference to the tileset for future visibility control
- Wait until the asset is fully loaded before interacting with it
- Apply any required height offset using a model matrix
- Optimise performance using:
 - maximumScreenSpaceError
 - dynamicScreenSpaceError
- Fly the camera to the loaded asset
- Log successful loading and errors

Create this as:

Section 5 - Photogrammetry Tileset

Do not modify existing sections.

This is my asset ID: `const assetIds = [4576801, 4618928];`

Prompt 4 – Load Multiple Tilesets & Align Scene

Extend the existing Cesium Sandcastle application.

Requirements:

- Load multiple Cesium ion tilesets from a list of asset IDs
- Add all tilesets to the scene
- Wait for all tilesets to finish loading
- Calculate a combined bounding sphere
- Fly the camera to the combined extent
- Log successful loading of each asset
- Log when all tilesets have finished loading

Store references for later interaction.

Create this as:

Section 6 - Tileset Management

Do not rewrite earlier sections.

Prompt 5 – Add Layer Control Widget

Extend the existing Cesium Sandcastle application.

Create a floating control panel.

Requirements:

- Create the panel dynamically using JavaScript
- Position the panel in the top-right corner
- Use a semi-transparent dark background
- Use a clean vertical layout
- Use reusable UI styling

Add the following checkboxes:

1. OSM Buildings
2. Photogrammetry

Behaviour:

- Each checkbox must toggle visibility of its corresponding layer
- Visibility changes should work even if assets load asynchronously
- Log all toggle actions

Create this as:

Section 7 - Layer Control Widget

Do not modify previous sections.

Prompt 6 – Improve User Experience

Extend the existing Cesium Sandcastle application.

Improve usability.

Requirements:

- Add clear comments throughout new code
- Improve console logging
- Add defensive checks before toggling assets
- Ensure the UI panel remains responsive during asset loading
- Display sensible default states for all checkboxes

- Follow the same coding style as existing sections

Create this as:

Section 8 - User Experience Improvements

Do not modify existing functionality.

Prompt X – Scaling Prompt (Take home exercise)

Extend the existing Cesium Sandcastle application with a new feature.

Rules:

- Do not rewrite or remove existing sections
- Add the feature as a clearly numbered new section
- Reuse existing variables and helpers where possible
- Follow the existing coding style
- Keep the solution modular
- Include console logging
- Ensure compatibility with Cesium Sandcastle

The new feature is:

(Describe your feature here)